

EXPERIMENT 11

11. AIM: Write a Program to implement Distance vector routing algorithm by obtaining routing table at each node (Take an example subnet graph with weights indicating delay between nodes).

SOLUTION:

Algorithm: Distance Vector Routing

Step 1: Start.

Step 2: Initialize the cost matrix with the **delay/cost between every pair of directly connected nodes**.

Step 3: For each node, initialize its routing table:

- Cost to itself = **0**
- Cost to a directly connected node = **link cost**
- Cost to an unreachable node = ∞

Step 4: Each node exchanges its **distance vector (routing table)** with its neighboring nodes.

Step 5: For each destination, calculate the new cost through each neighboring node using:

New Cost = Cost to Neighbor + Neighbor's Cost to Destination

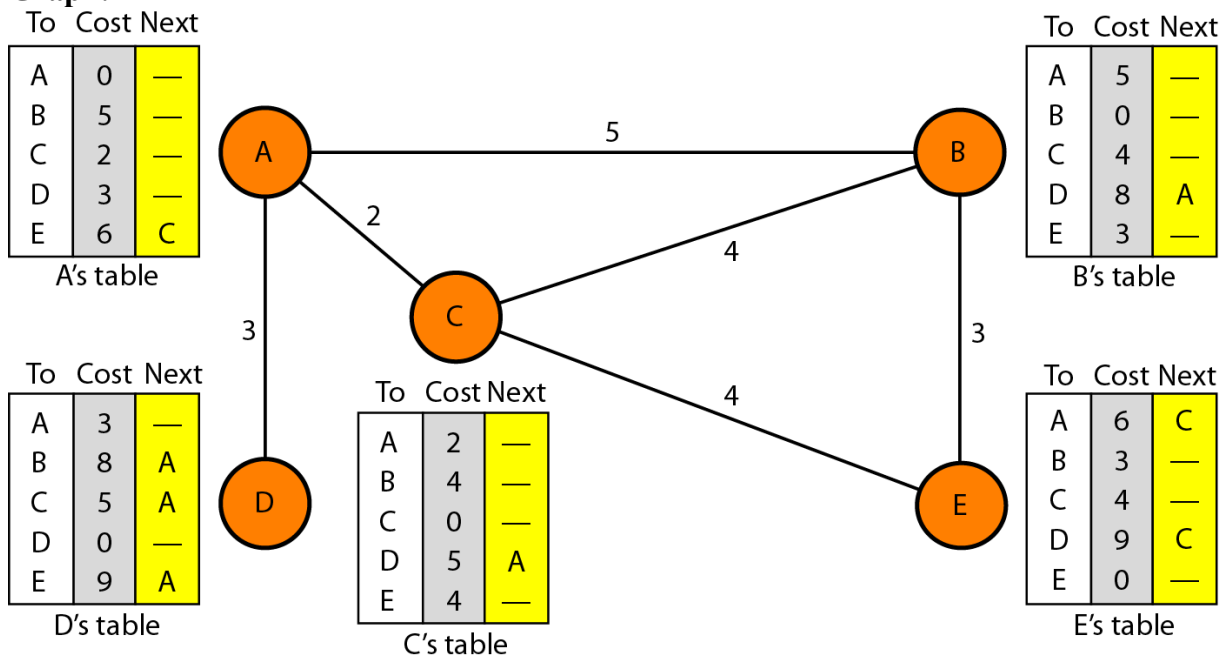
Step 6: If the new cost is smaller than the current cost, **update the cost and next hop** in the routing table.

Step 7: Repeat Steps 4–6 until **no routing table is changed**.

Step 8: Display the **routing table of each node**, showing destination, cost, and next hop.

Step 9: Stop.

Graph:



Source Code:

```
#include <stdio.h>
```

```
#define INF 9999
```

```
#define N 5
```

```
int main()
```

```
{
```

```
    int cost[N][N] = {
        /* A    B    C    D    E */
```

```

        { 0, 5, 2, 3, INF },
        { 5, 0, 4, INF, 3 },
        { 2, 4, 0, INF, 4 },
        { 3, INF, INF, 0, INF },
        { INF, 3, 4, INF, 0 }
};

int dist[N][N];
int next[N][N];
int i, j, k;
int changed;

/* Initialize routing tables */
for (i = 0; i < N; i++)
{
    for (j = 0; j < N; j++)
    {
        dist[i][j] = cost[i][j];
        next[i][j] = -1;
    }
}

/* Distance Vector Algorithm */
do
{
    changed = 0;

    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N; j++)
        {
            for (k = 0; k < N; k++)
            {
                if (cost[i][k] != INF &&
                    dist[k][j] != INF &&
                    dist[i][j] > cost[i][k] + dist[k][j])
                {
                    dist[i][j] = cost[i][k] + dist[k][j];

                    if (next[i][k] == -1)
                        next[i][j] = k;
                    else
                        next[i][j] = next[i][k];

                    changed = 1;
                }
            }
        }
    }
} while (changed);

/* Display routing tables */
for (i = 0; i < N; i++)
{

```

```

printf("\n%c's Routing Table\n", 'A' + i);
printf("To\tCost\tNext\n");

for (j = 0; j < N; j++)
{
    printf("%c\t%d\t", 'A' + j, dist[i][j]);

    /* Show - for directly connected nodes */
    if (i == j || cost[i][j] != INF)
        printf("-\n");
    else
        printf("%c\n", 'A' + next[i][j]);
}

return 0;
}

```

Output: